

★ WEEK 14 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Dissolve a Signal · Compute a Shapley Value · Measure a Gap

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the prompts generate data; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Confounding	10-day table on the slides	fml_w14_p1_days.csv (hand-calc)
P2 · Shapley	four coalition values on the slides	— (2 features, no file needed)
P3 · Fairness gap	group counts on the slides	fml_w14_p3_groups.csv (hand-calc)
Scaled labs	\$ 5 in-script set (make_classification)	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · CONFOUNDING

Dissolve the Buzz Signal

Ten days, one hidden regime — stratify and watch 1.2pp evaporate

'Social Buzz Predicts Returns' — Does It?

Ten trading days. Five were rally days (+2% each), five calm (0%). Buzz was high on 4 rally days and 1 calm day

Day	1	2	3	4	5	6	7	8	9	10
Regime	rally	rally	rally	rally	rally	calm	calm	calm	calm	calm
Buzz	H	H	H	H	L	H	L	L	L	L
Return	+2	+2	+2	+2	+2	0	0	0	0	0

Your deliverables

- The naive comparison: average return on buzz-H days vs buzz-L days.
- The same comparison WITHIN rally days only, and WITHIN calm days only.
- A one-line verdict on the buzz signal, using the word 'confounder'.

The Tool: Compare Within, Not Across

If a hidden factor drives both X and Y, stratify by it — the honest gap lives inside the strata

$$\text{Naive gap} = \bar{Y}_{\text{buzz}} - \bar{Y}_{\text{quiet}}$$

Controlled gap: compare ONLY within the same regime

Why the naive gap lies

- Rally days have MORE buzz and HIGHER returns — buzz inherits the credit.
- The comparison mixes regimes: H-days are mostly rally, L-days mostly calm.
- That composition difference IS the 1.2pp.

Lab guide

- Step 1: averages for H (5 days) and L (5 days) overall.
- Step 2: repeat inside rally days only.
- Step 3: repeat inside calm days only; compare all three gaps.

One Fake Gap, Two Honest Zeros

The whole 'signal' was the regime wearing a buzz costume

Comparison	Buzz-H average	Buzz-L average	Gap
Naive (all 10)	$(2+2+2+2+0)/5 = 1.6\%$	$(2+0+0+0+0)/5 = 0.4\%$	+1.2pp
Within rally (5 days)	$(2+2+2+2)/4 = 2.0\%$	$2/1 = 2.0\%$	0
Within calm (5 days)	$0/1 = 0\%$	$0/4 = 0\%$	0

Naive: $1.6\% - 0.4\% = 1.2\text{pp}$ Within rally: 0 Within calm: 0 (all confounder)

1.2pp of Signal, 0pp of Cause

Trade on buzz and you are really trading on 'is it a rally?' — which you learn too late

+1.2pp

The naive gap — real arithmetic, fake meaning

0 & 0

Both within-regime gaps — the signal dissolved

4 / 5

H-days that were rally days — the composition trick

rung 1

Where this evidence lives on the causal ladder

The general lesson

- Any factor correlated with the regime inherits its returns.
- Week 4's factor zoo is thousands of buzz signals in better suits.

What would upgrade it

- An experiment: randomly amplify buzz? Impossible here.
- So: stratify, control, and hold the conclusion loosely.

Scale It: Manufacture and Dissolve Confounders

Paste the metaprompt — Claude Code builds fake signals and teaches you to kill them

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: the 10-day table -> naive  
   gap 1.2pp, within-regime gaps 0 and 0;  
   (reference: fml_w14_p1_days.csv)  
2) simulates 2,000 days: a hidden regime (rally 30% of  
   days) adds +1% to returns AND raises P(buzz high)  
   from 0.2 to 0.8; buzz itself has ZERO effect;  
3) prints the naive buzz-return gap with a t-stat -  
   show it looks 'highly significant';  
4) stratifies by regime and prints both within-regime  
   gaps - show they are ~0;  
5) bonus: regress return on buzz AND regime together;  
   show buzz's coefficient collapses;  
6) three-sentence verdict on what 'controlling for a  
   confounder' means, in this week's language.
```

[OUTPUT] What to expect

```
toy check: 1.2pp / 0 / 0  
naive gap: ~1pp with a  
   seductive t-stat  
within-regime: ~0 both  
regression: buzz coef ~0  
   once regime enters
```

Your 10-day trick, played
on 2,000 days - and
caught the same way.

P2

PROBLEM 2 · SHAPLEY

One Prediction, Fairly Divided

Two clues, two arrival orders — game theory with a pencil

Explain One Applicant's Score

A default model. Average prediction 0.40. For applicant A: with Income only 0.30, Debt only 0.70, both 0.65

Coalition of clues	Model output $f(\cdot)$
$\emptyset$ (no clues — the base)	0.40
{Income}	0.30
{Debt}	0.70
{Debt, Income} (the prediction)	0.65

The Shapley rule

- ϕ = the marginal push, averaged over the two arrival orders.

Your deliverables

- ϕ_{Debt} , ϕ_{Income} , and the additivity check to 0.65.

The Tool: Average the Marginal Pushes

'What did you add, given who was already in the room?' — asked in every order, then averaged

$$\phi_D = \frac{1}{2} [f(D) - f(\emptyset)] + \frac{1}{2} [f(D, I) - f(I)] \quad (\text{average over both arrival orders})$$

Why average over orders

- A clue's push depends on who arrived first — Debt looks bigger alone.
- Averaging removes the arrival-order luck: fairness by construction.
- With d features there are $d!$ orders; SHAP handles the explosion for you.

Lab guide — per clue

- Step 1: push when it arrives FIRST (vs the base).
- Step 2: push when it arrives SECOND (vs the other alone).
- Step 3: average; then verify $\text{base} + \phi_D + \phi_I = f(D, I)$.

Four Marginals, Two Averages, One Check

Nothing left unexplained — the parts sum exactly to the prediction

Clue	Arrives first	Arrives second	ϕ (average)
Debt	$0.70 - 0.40 = +0.30$	$0.65 - 0.30 = +0.35$	$+0.325$
Income	$0.30 - 0.40 = -0.10$	$0.65 - 0.70 = -0.05$	-0.075
Additivity	$0.40 + 0.325 - 0.075$	$= 0.65$	$= f(D, I) \checkmark$

$$0.40 \text{ (base)} + \phi_D 0.325 - \phi_I 0.075 = 0.65 \checkmark \quad (\text{adds up exactly})$$

Debt Pushed +0.325, Income Pulled −0.075

The customer letter writes itself — and every cent of the score is accounted for

+0.325

Debt's fair share of the move from 0.40 to 0.65

−0.075

Income's pull — protective, and quantified

= 0.65

Base plus both ϕ 's, exactly — the additivity guarantee

d! orders

What real SHAP averages over when d is large

Why 'alone' $\neq \phi$

- Debt alone pushed +0.30, but +0.35 after Income — interaction.
- The average is the only split that stays fair either way.

The Week-14 caution

- ϕ explains the MODEL's arithmetic, not the world's causes.
- 'Debt drove the score' $\neq$ 'paying debt changes true risk' — that's P1's ladder.

Scale It: From Two Clues to a Real Explainer

Paste the metaprompt — Claude Code verifies your ϕ 's and explains a trained model

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: base 0.40, f(I)=0.30,  
   f(D)=0.70, f(D,I)=0.65 -> phi_D = 0.325,  
   phi_I = -0.075, additivity to 0.65;  
2) generates the class dataset (make_classification,  
   20000 x 20, weights [0.99, 0.01], random_state=42),  
   trains a RandomForest;  
3) implements exact 2-feature Shapley on the model's  
   top-2 features (marginalize the rest by sampling):  
   explain 3 individual test rows, printing base, each  
   phi, and the additivity check;  
4) compares: permutation importance ranking vs the  
   average |phi| ranking over 200 rows;  
5) three-sentence verdict: what per-row explanation  
   adds over global importance, in this week's  
   language.
```

[OUTPUT] What to expect

```
toy check: 0.325 / -0.075  
           exactly  
3 rows: signed phi per  
        clue, sums verified  
rankings: broadly agree,  
          disagree on interactions  
  
Your two-order average,  
grown into an explainer  
for 20,000 rows.
```

P3

PROBLEM 3 · FAIRNESS

Two Groups, One Scale

Same talent pool, different odds — measure it, name it, gate it

An Approval Model Meets Two Groups

100 applicants per group, 80 qualified in EACH. The model approved only qualified people — but not the same share

	Group A	Group B
Applicants	100	100
Qualified	80	80
Approved (all qualified)	60	40
Rejected qualified (FN)	20	40
Rejected unqualified (TN)	20	20

Your deliverables

- Approval rate, TPR, and accuracy per group; the parity gap and the opportunity gap — and why overall accuracy hid it.

The Tool: Two Gaps, Two Different Harms

Parity asks 'who gets approved?'; opportunity asks 'who gets approved AMONG the deserving?'

$$\text{Parity gap} = P(\text{approve} \mid A) - P(\text{approve} \mid B) \quad \text{Opportunity gap} = \text{TPR}_A - \text{TPR}_B$$

Reading the metrics

- Parity gap: raw approval rates — visible to everyone outside.
- Opportunity gap (TPR): odds GIVEN you deserved it — the deeper injury.
- The metrics can disagree; choose by which harm matters here.

Lab guide — per group

- Step 1: approval rate = approved / 100.
- Step 2: TPR = approved / qualified.
- Step 3: accuracy = (TP + TN) / 100; then take both gaps.

Six Ratios, Two Gaps

Identical talent pools — the model still treated them differently

Metric	Group A	Group B	Gap
Approval rate	$60 / 100 = 60\%$	$40 / 100 = 40\%$	20pp (parity)
TPR (qualified only)	$60 / 80 = 75\%$	$40 / 80 = 50\%$	25pp (opportunity)
Accuracy	$(60+20)/100 = 80\%$	$(40+20)/100 = 60\%$	20pp

$$\text{Parity: } 60\% - 40\% = 20\text{pp}$$

$$\text{Opportunity: } 75\% - 50\% = 25\text{pp}$$

A 25pp Opportunity Gap on Equal Talent

No false approvals anywhere — and still profoundly unfair. Precision alone can't see this

20pp

Parity gap — the difference the outside world sees

25pp

Opportunity gap — the injury to the equally qualified

70%

Blended overall accuracy — the number that hid everything

0 FP

Both groups: the model was 'careful' and unfair at once

Where gaps come from

- Thresholds tuned on one group's data; proxy features; thin history.
- None require any intent — audits exist because intent is irrelevant.

The deployment gate

- Report both gaps with every model card; set tolerance in advance.
- A gap this size = fix before launch, with humans owning the call.

Scale It: a Full Per-Group Audit

Paste the metaprompt — Claude Code audits a trained model the way a reviewer would

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: the A/B counts -> approval  
   60%/40%, TPR 75%/50%, accuracy 80%/60%; print the  
   parity (20pp) and opportunity (25pp) gaps;  
   (reference: fml_w14_p3_groups.csv)  
2) generates the class dataset (make_classification,  
   seed 42), defines group = (feature 0 > median),  
   trains a RandomForest;  
3) audits per group: approval rate, TPR, FPR,  
   accuracy - one tidy table with both gaps;  
4) threshold repair: sweep the decision threshold per  
   group to equalize TPRs; report the new gaps and  
   what it cost in overall accuracy;  
5) three-sentence verdict: which gap you would gate  
   on for lending, and why, in this week's language.
```

[OUTPUT] What to expect

```
toy check: 20pp / 25pp  
           exactly  
audit table: per-group  
             rates, gaps flagged  
repair: TPR gap ~0 at a  
        small accuracy price  
  
Your six ratios, promoted  
to a launch-review audit.
```

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

1

The buzz signal: 1.2pp naive, 0 within each regime

The regime caused both buzz and returns — stratify by the confounder and the 'signal' dissolves entirely.

2

Shapley: $\phi_D = +0.325$, $\phi_I = -0.075$, sums to 0.65 exactly

Average each clue's push over arrival orders — fair by construction, and nothing left unexplained.

3

Fairness: 20pp parity, 25pp opportunity, on equal talent

Zero false approvals and still unfair — only per-group metrics see it, and they gate the launch.

Hand calculation first — then let Claude Code scale the same idea to real data.